Software Defined Networking — Lab 2

Boubacar Diallo

October 2025

Exercise 1 — Q1: Simple Tree Topology (High-Level API)

File: `simpletree_highlevel.py`

The script builds an n-ary tree topology: one core switch `c1`, n aggregation switches (`a1..an`), each aggregation switch connected to n edge switches (`e_i_j`), and each edge switch serving n hosts.

Commands used:

```
sudo python3 simpletree_highlevel.py -n 2
mininet> pingall
```

Observation: end-to-end reachability is correct when all switches are up.

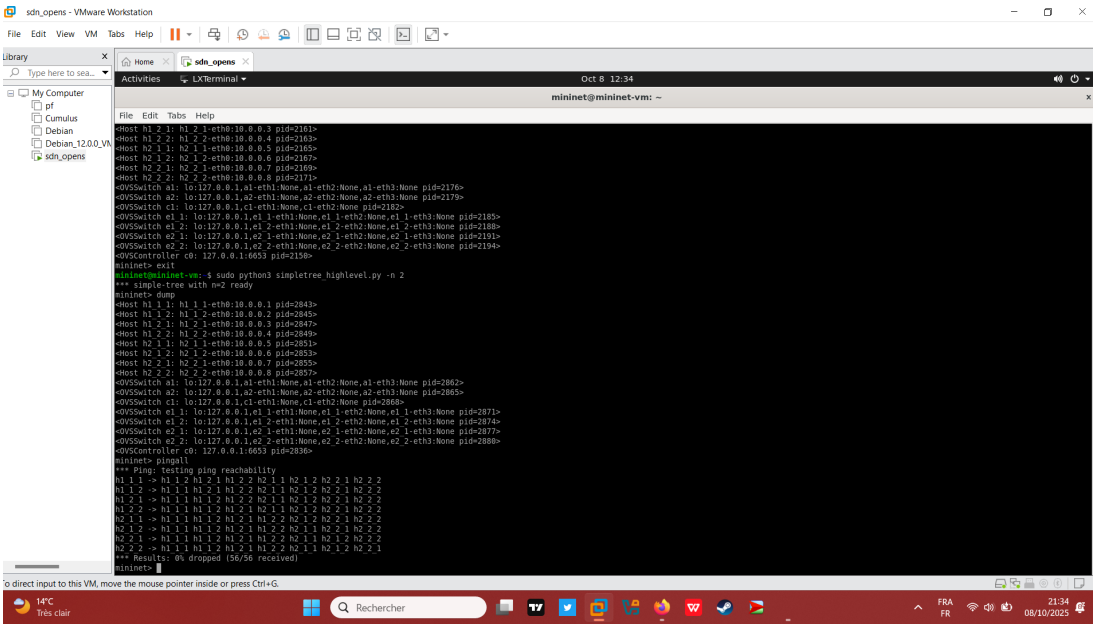

Figure 1: `pingall` connectivity test (0% packet loss)

The `0% packet loss` result confirms that all host pairs can communicate and the topology was instantiated correctly.

Exercise 1 — Q2: Simple Tree (Low-Level API)

File: `simpletree_lowlevel.py`. This version explicitly creates all nodes and links (core, aggregation, edge, hosts).

```
sudo python3 simpletree_lowlevel.py -n 2
mininet> pingall
```

Observation: same result as Q1 (full connectivity).

Figure 2: `pingall` result with the low-level topology

High-Level vs. Low-Level API

Both scripts build the exact same topology but follow different approaches.

1. High-Level API (Q1). We describe the network structure declaratively in a `Topo` subclass. Mininet then instantiates hosts, switches, and links. It's concise and readable.

2. Low-Level API (Q2). We build the network imperatively, adding every host, switch, and link ourselves. We gain full control at the expense of verbosity.

Summary. Use the high-level API for fast, readable definitions of fixed topologies; use the low-level API when you need fine-grained control or special configurations.

Exercise 1 — Q3: Core Switch Failure Test

We keep the previous tree topology (`n = 2`) and stop the core switch `c1`, which interconnects all branches.

Commands used:

```
sudo python3 simpletree_highlevel.py -n 2
mininet> pingall                # Baseline: everything works
mininet> switch c1 stop         # Stop the core switch
mininet> pingall                # Test again
```

Observation:

- Before stopping `c1`, all hosts can reach each other (0% loss).
- After `switch c1 stop`, inter-branch communication breaks: hosts under the same aggregation switch remain reachable locally, but no traffic crosses the tree.

Figure 3: Connectivity after stopping the core switch `c1`

Why this happens. The core switch is a single point of transit. When it fails, branches become isolated— a textbook *single point of failure*.

Exercise 2 — L2 Loop and Spanning Tree Protocol (STP)

We create a layer-2 loop among three switches (`s1`, `s2`, `s3`) and observe behavior with STP disabled, then enabled.

Steps:

1. Start POX:

   ```
   cd ~/pox
   ./pox.py forwarding.l2_learning openflow.of_01 --port=6633
   ```

2. Start the looped topology:

   ```
   sudo mn --custom topo_loop.py --topo loop \
     --controller=remote,ip=127.0.0.1,port=6633 \
     --switch ovsk --mac
   ```

3. Check connectivity:

   ```
   mininet> h1 ping -c10 10.0.0.2
   ```

   The ping fails with `Destination Host Unreachable` due to the forwarding loop (STP off).

4. Check STP status:

   ```
   sudo ovs-vsctl list bridge | grep stp_enable
   ```

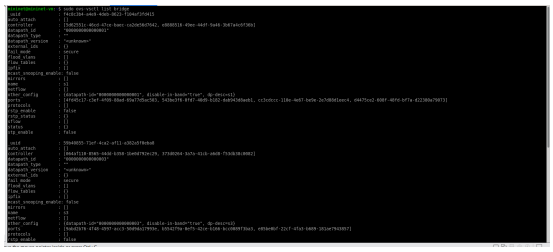

Figure 4: Left: STP disabled

5. Enable STP:

```
sudo ovs-vsctl set bridge s1 stp_enable=true
sudo ovs-vsctl set bridge s2 stp_enable=true
sudo ovs-vsctl set bridge s3 stp_enable=true
```

6. Test again:

```
mininet> h1 ping -c5 10.0.0.2
```

This time communication is restored: STP blocks one link to break the loop.

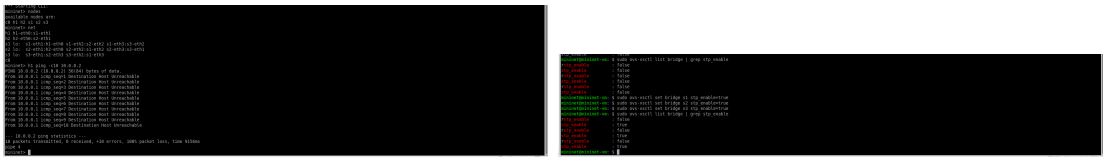

Figure 5: Left: ping fails with STP off; right: connectivity restored with STP on

Conclusion. Without STP, frames loop indefinitely (broadcast storm) and connectivity collapses. With STP on, one link is blocked, the topology stabilizes, and ping succeeds.

1 Exercise 3 — Inspecting OpenFlow with Wireshark

1.1 Goal

Observe the message exchange between the OpenFlow controller (**Ryu**) and the virtual switch (**Open vSwitch**) using Wireshark to understand how communication is established and flow rules are installed.

1.2 Environment Preparation

Verify installed tools

```
sudo apt update
sudo apt install mininet openvswitch-switch ryu-manager wireshark -y
```

Allow capture on interface `lo`

```
sudo groupadd wireshark
sudo dpkg-reconfigure wireshark-common   # Answer "Yes"
sudo usermod -aG wireshark $USER
groups $USER   # Check that "wireshark" appears
```

After re-login, launch Wireshark:

```
wireshark &
```

Select interface `lo` and apply the filter:

```
tcp.port == 6633 || tcp.port == 6653
```

1.3 Start the Controller and Mininet

Start Ryu:

```
ryu-manager --verbose ryu.app.simple_switch_13
```

Start Mininet:

```
sudo mn -c
sudo mn --topo single,2 --mac \
  --switch ovsk,protocols=OpenFlow13 \
  --controller=remote,ip=127.0.0.1,port=6633
```

This topology has two hosts (`h1`, `h2`), one switch (`s1`), and a remote controller (Ryu).

1.4 Observed OpenFlow Messages

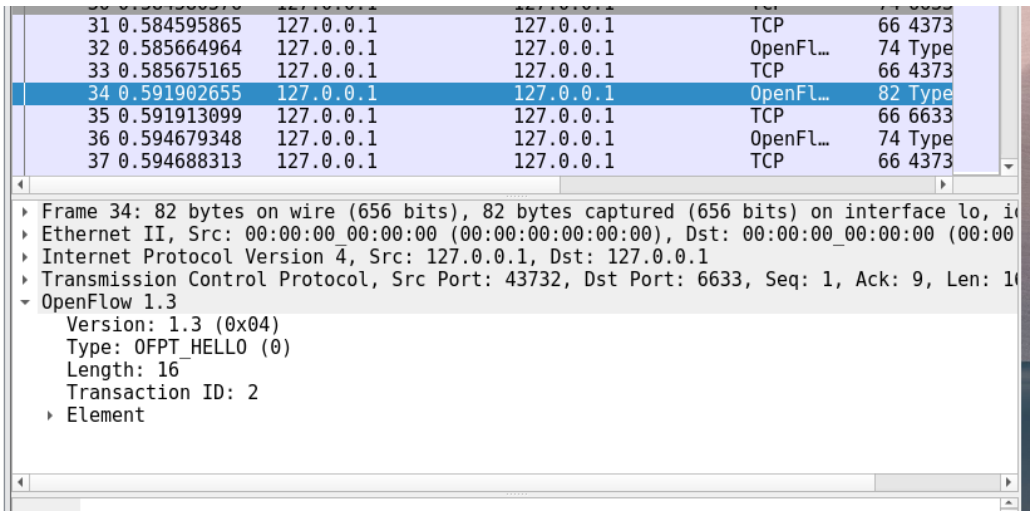

Figure 6: `HELLO` — version negotiation between controller and switch

(a) `HELLO` Both entities exchange `HELLO` messages to agree on the OpenFlow version (1.3 here).

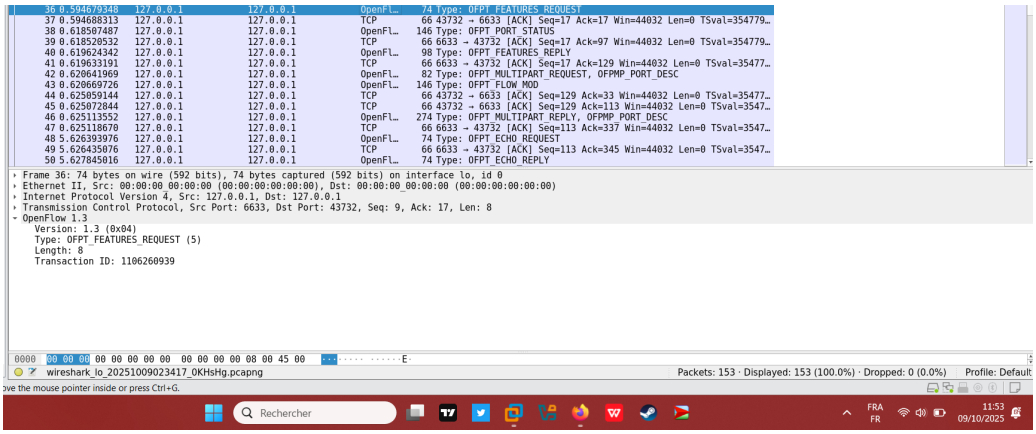

Figure 7: `FEATURES_REQUEST` / `REPLY` exchange

(b) `FEATURES_REQUEST` / `REPLY` The controller queries the switch for its characteristics (datapath ID, ports, capabilities).

(c) `SET_CONFIG` The controller configures, for example, the maximum packet bytes to send up for unmatched packets.

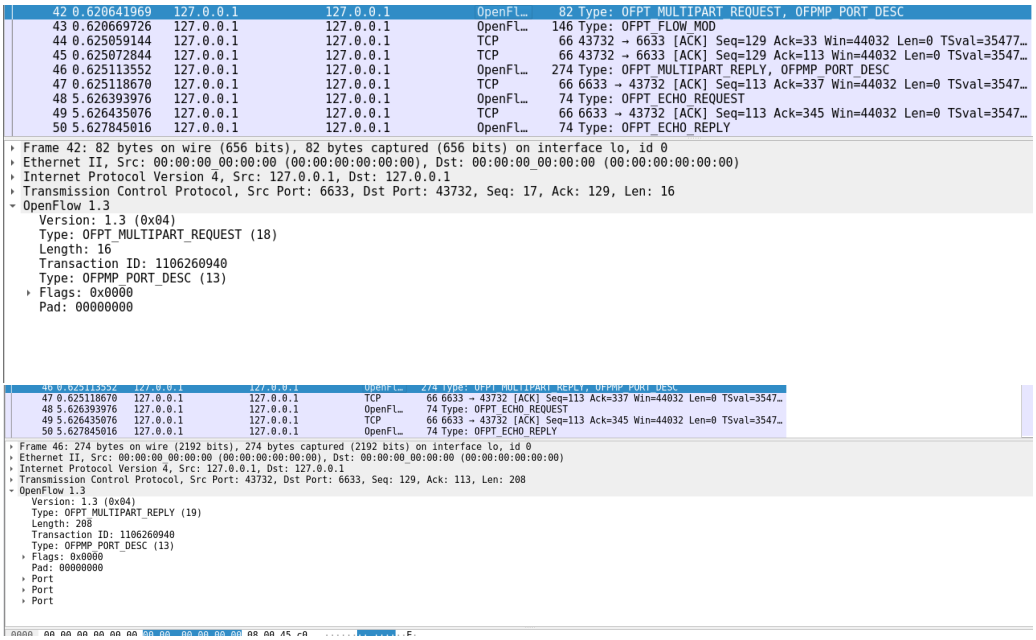

Figure 8: Statistics requests between controller and switch

(d) `MULTIPART_REQUEST` / `REPLY` The controller retrieves port and flow-table statistics.

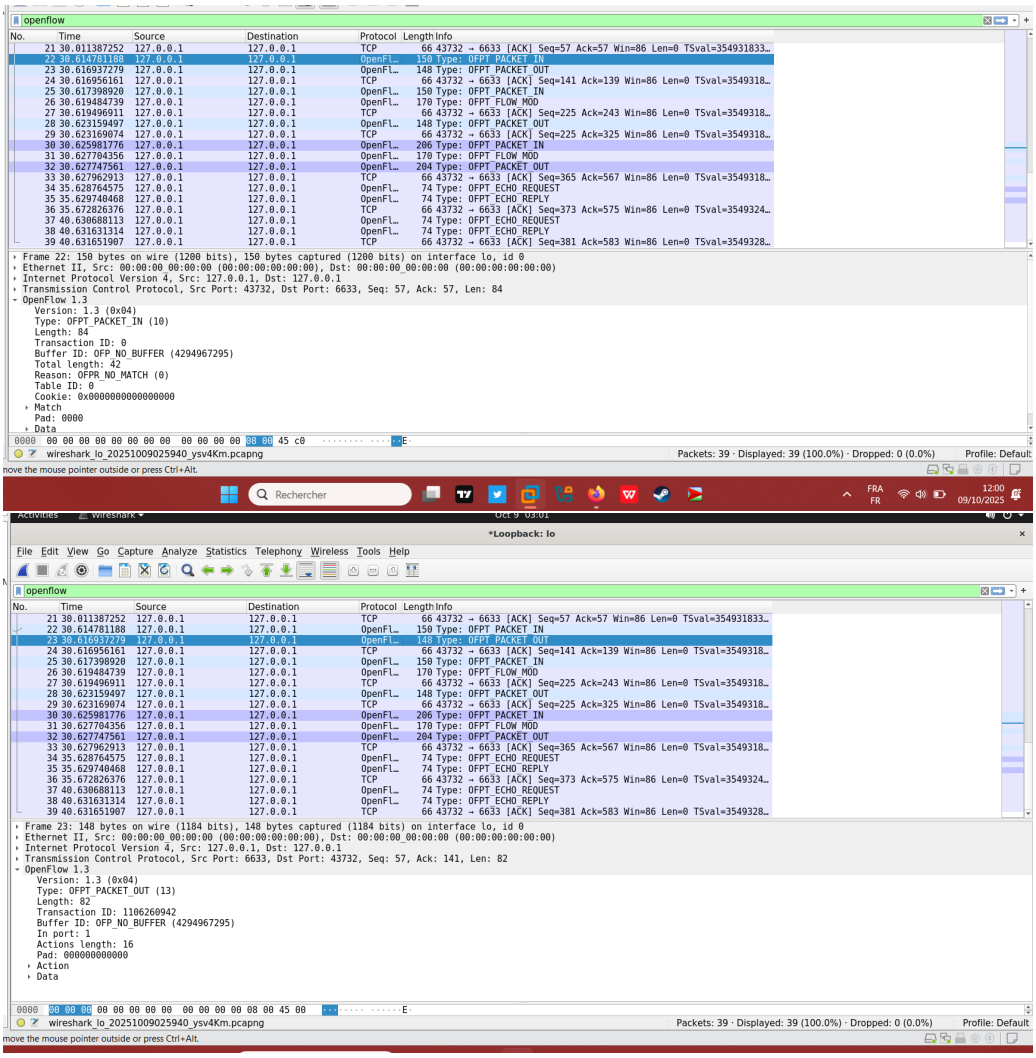

Figure 9: `PACKET_IN`: the switch punts an unknown packet to the controller

(e) `PACKET_IN` At the first `ping` between `h1` and `h2`, the switch has no matching flow, so it sends the ICMP packet to the controller for a decision.

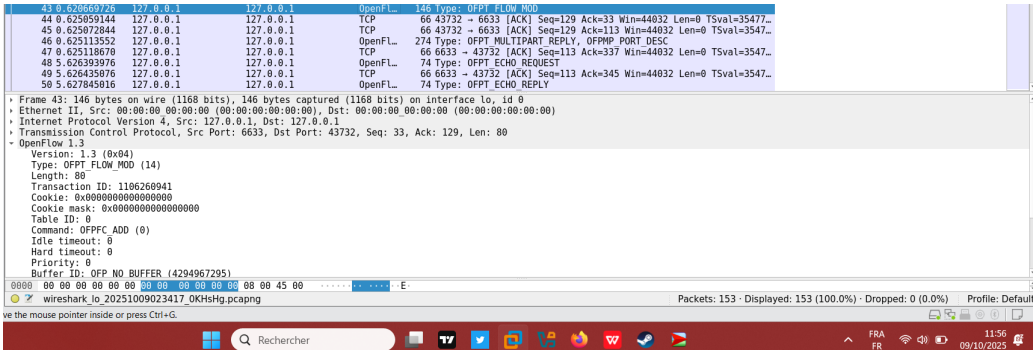

Figure 10: `FLOW_MOD` — installing a rule in the switch

(f) `FLOW_MOD` The controller installs rules such as:

- If src=`h1`, dst=`h2` → output `s1-eth2`

- If src=`h2`, dst=`h1` → output `s1-eth1`

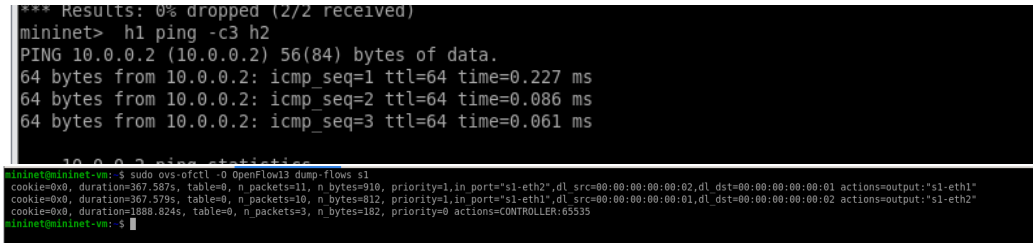

Figure 11: Ping succeeds once flows are installed; link-table counters increase

(g) Successful ping Once the flows are in place, subsequent pings go directly switch-to-switch, without involving the controller.

(h) Deleting flows

```
sudo ovs-ofctl -O OpenFlow13 del-flows s1
```

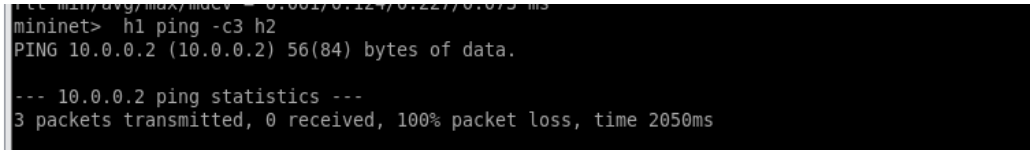

Figure 12: Ping fails after removing flows

Without flows, the switch no longer knows where to forward packets (`Destination Host Unreachable`).

(i) Restoring the default punt-to-controller flow

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=0,actions=CONTROLLER:65535"
```

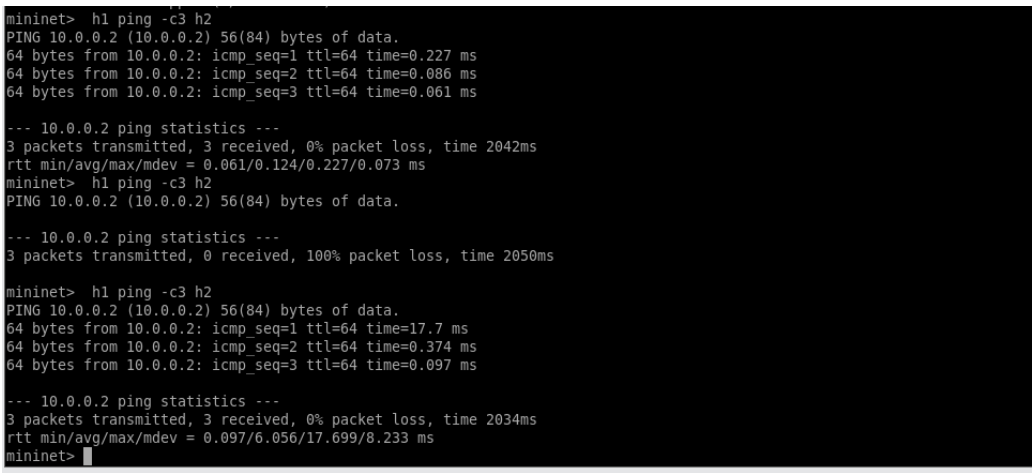

Figure 13: Controller-switch path restored

This rule tells the switch to send unknown packets to the controller again, restoring communication.

1.5 OpenFlow Message Sequence (Summary)

Step	Observed OpenFlow Message
1	HELLO — version negotiation
2	FEATURES_REQUEST / REPLY — switch identification
3	SET_CONFIG — initial configuration
4	MULTIPART — port/flow statistics
5	PACKET_IN — unknown packet sent to controller
6	FLOW_MOD — rule installation
7	PING OK — direct data plane forwarding
8	DEL_FLOWS — rules removed
9	ADD_FLOW — controller path restored

This experiment illustrates OpenFlow's control/data-plane split: the controller makes decisions, the switch enforces rules, and Wireshark shows the dialogue.

2 Exercise 4 — Manipulating Flows with `dpctl`

Objective

Understand how an OpenFlow switch manages flows and how to manipulate them manually using `dpctl`/`ovs-ofctl`, without an external controller.

Step 1 — Build the Topology

Create one switch `s1` and three hosts (`h1`, `h2`, `h3`) with no controller:

```
sudo mn -c
sudo mn --topo single,3 --mac --controller=none --switch ovsk,protocols=OpenFlow13
```

Output:

```
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
```

Observation: no connectivity by default (no flows installed).

Step 2 — Enable the OpenFlow listen port

Allow `dpctl` to connect on TCP `6634`:

```
sudo ovs-vsctl set-controller s1 ptcp:6634
sudo ovs-vsctl get-controller s1
# Expected: ptcp:6634
```

Step 3 — Check OpenFlow port numbers

```
sudo ovs-ofctl -O OpenFlow13 show s1
```

Typical mapping:

```
1(s1-eth1): h1
2(s1-eth2): h2
3(s1-eth3): h3
```

Step 4 — Baseline connectivity

```
mininet> h1 ping -c3 h2
# Destination Host Unreachable
```

Step 5 — Add flows manually

Allow traffic between `h2` and `h3`:

```
dpctl del-flows tcp:127.0.0.1:6634
dpctl add-flow tcp:127.0.0.1:6634 in_port=2,actions=output:3
dpctl add-flow tcp:127.0.0.1:6634 in_port=3,actions=output:2
dpctl dump-flows tcp:127.0.0.1:6634
```

Step 6 — Verify

```
mininet> h2 ping -c3 h3
```

Observation: ping succeeds (0% loss). Packet/byte counters increase:

```
dpctl dump-flows tcp:127.0.0.1:6634
# n_packets and n_bytes > 0
```

Figure 14: Ping succeeds after adding flows (h2 ↔ h3)

Step 7 — Remove flows and test again

```
dpctl del-flows tcp:127.0.0.1:6634
mininet> h2 ping -c3 h3
# Destination Host Unreachable
```

Conclusion: without flows, no packets are forwarded.

Q&A

- **Q1.** Does host-to-host ping work without a controller? *No.* The switch has an empty flow table.
- **Q2.** After adding `in_port=2` ↔ `3` flows? *Yes.* `h2` ↔ `h3` ping succeeds.
- **Q3.** After deleting flows? *Ping fails* again because the switch has no forwarding rule.
- **Q4.** What do `n_packets`/`n_bytes` show? They are per-rule counters for matched traffic.

Takeaway

Flow rules are essential in SDN. With no controller and no flows, there is no traffic. Manually adding flows with `dpctl` emulates what an SDN controller (POX/Ryu) would automate.

3 Exercise 5 — Topology Discovery with ONOS

I deployed **ONOS 1.15.0** on Debian and connected it to a Mininet host on Ubuntu to visualize automatic SDN topology discovery.

1. Environment and Network Setup

Two VMs:

- **Debian (192.168.81.194)**: ONOS controller
- **Ubuntu (192.168.81.195)**: Mininet emulator

Both are on the same LAN. ONOS listens on 6653 (OpenFlow) and 8181 (GUI).

2. Install Java 8 and ONOS

```
sudo apt update
sudo apt install openjdk-8-jdk -y
java -version   # openjdk version "1.8.0_XXX"
```

Download and start ONOS:

```
wget https://repo1.maven.org/maven2/org/onosproject/onos-releases/1.15.0/onos-1.15.0.t
tar -xvf onos-1.15.0.tar.gz
cd onos-1.15.0
./bin/onos-service start
```

Verify ports:

```
ss -lntp | grep 6653
ss -lntp | grep 8181
```

3. Connect Mininet to ONOS

On Ubuntu:

```
sudo mn --controller=remote,ip=192.168.81.194,port=6653 \
  --topo linear,5 --switch ovsk,protocols=OpenFlow13
```

```
mininet> pingall
```

All hosts reach each other, indicating ONOS `fwd` is working.

4. Inspect in ONOS

```
onos> devices
onos> hosts
onos> links
onos> topology
```

The five switches and five hosts appear. In the browser, open `http://192.168.81.194:8181/onos/ui` to view the live topology.

5. Screenshots

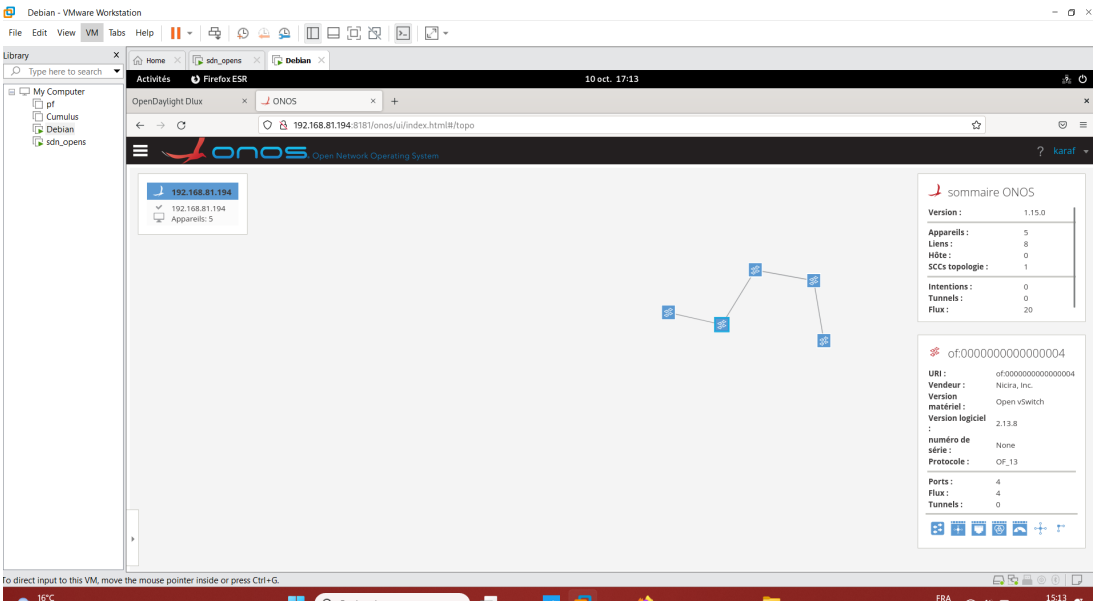

Figure 15: Topology discovered automatically by ONOS

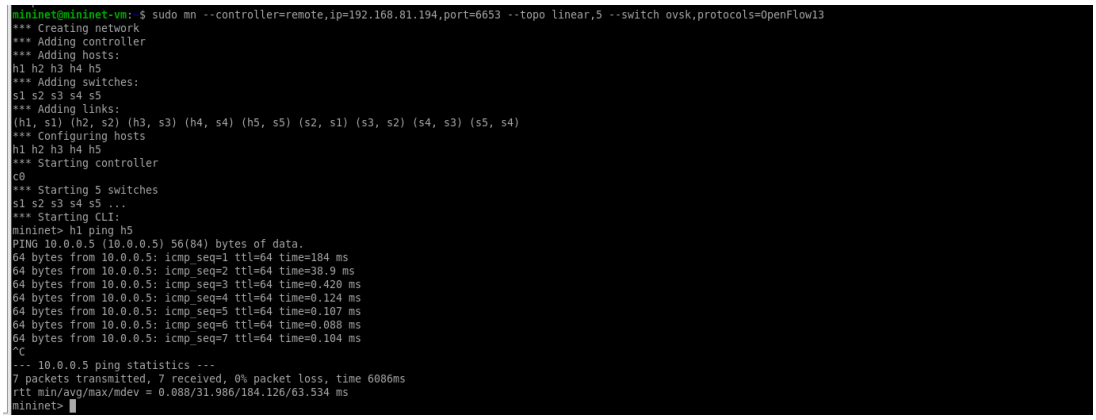

Figure 16: `pingall` validation through ONOS

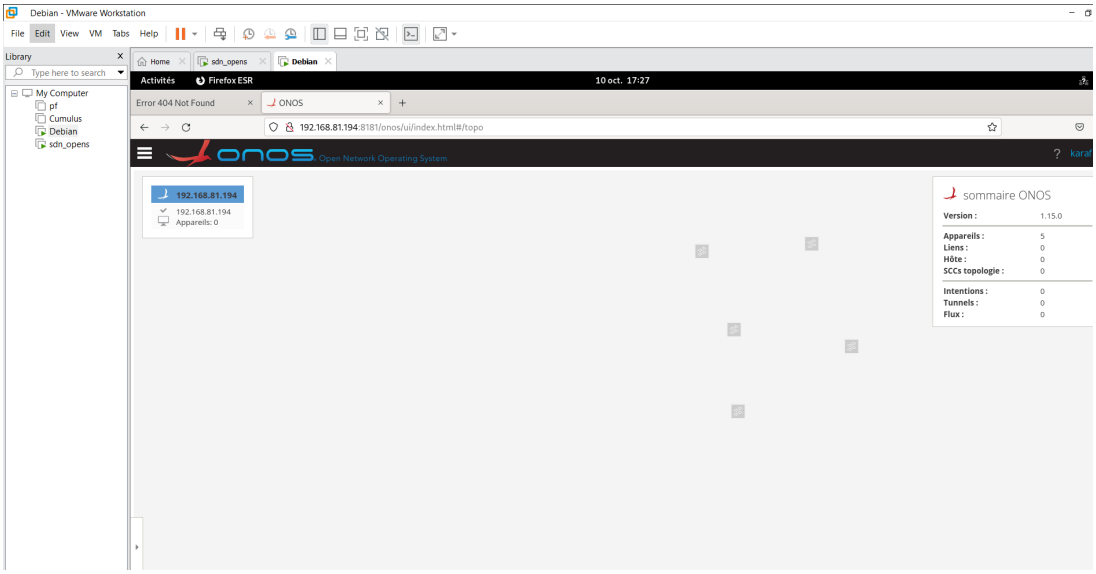

Figure 17: ONOS view after stopping the Mininet topology

6. Q&A

- **Discovery protocol.** ONOS uses **LLDP** and its broadcast variant **BDDP**.
- **Ethertype and MACs.** LLDP uses ethertype `0x88cc` and multicast MAC `01:80:C2:00:00:0E`; BDDP uses `FF:FF:FF:FF:FF:FF`.
- **OpenFlow messages.** Switches send `PACKET_IN` with LLDP to the controller; ONOS replies with `PACKET_OUT` to probe other links.
- **LLDP frequency.** Roughly every 5 seconds.
- **Process summary.** ONOS periodically emits LLDP/BDDP via OpenFlow ports; switches punt them to ONOS, which reconstructs the topology in real time.

7. Conclusion

With Java 8 + ONOS on Debian and Mininet on Ubuntu, we observed automatic topology discovery and validated connectivity. Relevant ONOS apps included `openflow`, `fwd`, `proxyarp`, `lldpprovider`, and `hostprovider`.

4 OpenDaylight (ODL)

Install and test **OpenDaylight Carbon** with **Mininet** in a linear topology of 5 hosts and 5 switches, then analyze LLDP and OpenFlow traffic with **Wireshark**.

5 Network Configuration

Two VMs:

- **ODL (Debian)**: `192.168.10.1`
- **Mininet**: `192.168.10.2`

Static IPs via Netplan:

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s8:
      dhcp4: no
      addresses: [192.168.10.1/24]
```

6 Install OpenDaylight (Carbon)

1. Download and extract:

```
cd /opt
sudo wget https://nexus.opendaylight.org/content/repositories/public/or
sudo tar -xzf distribution-karaf-0.6.4-Carbon.tar.gz
sudo mv distribution-karaf-* opendaylight
cd /opt/opendaylight
```

2. Start ODL:

```
bin/karaf
```

3. In Karaf, install features:

```
feature:install odl-restconf-all
feature:install odl-openflowplugin-southbound
feature:install odl-l2switch-all
feature:install odl-dluxapps-applications
```

4. Verify open ports:

```
sudo ss -tlnp | grep java
# Expected: 6653 (OpenFlow), 8181 (RESTCONF / DLUX)
```

7 Deploy the Mininet Topology

On the Mininet VM:

```
sudo mn --topo linear,5 \
  --controller=remote,ip=192.168.10.1,port=6653 \
  --switch ovsk,protocols=OpenFlow13
```

Verify:

```
mininet> pingall
mininet> sh ovs-vsctl show
```

8 DLUX GUI

Open:

`http://192.168.10.1:8181/index.html`

Credentials: `admin / admin`

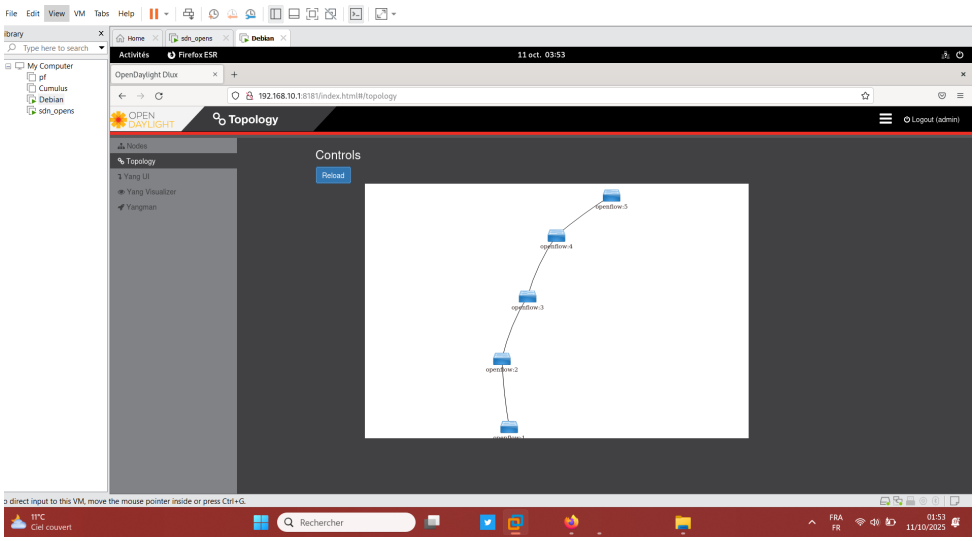

Figure 18: DLUX showing 5 switches and 5 hosts

9 Wireshark Analysis and Q&A

Q1 — LLDP filter in Wireshark

Use:

`!(ip.version==4)`

This hides IP packets to focus on L2 frames (including LLDP).

Q2 — Mininet command

```
sudo mn --topo linear,5 --controller=remote,ip=192.168.10.1,port=6653 --switch ovsk,p
```

Q3 — Show created switches

```
ovs-vsctl show
```

Q4 — Start the controller

ODL:

```
/opt/opendaylight/bin/karaf
```

ONOS:

```
/opt/onos/apache-karaf-<version>/bin/karaf
```

Q5 — Validate operation

In Mininet:

```
mininet> pingall
```

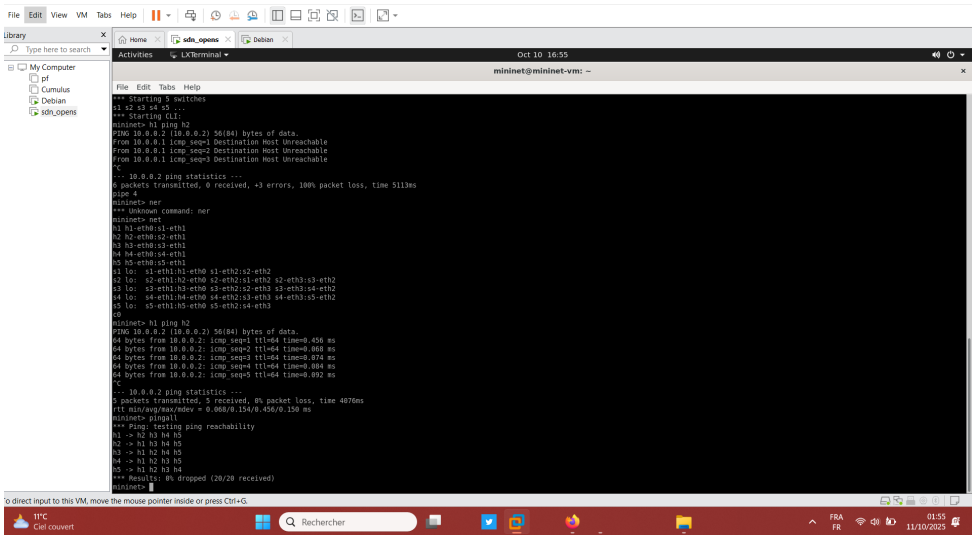

Figure 19: `pingall` result

Q6 — LLDP addressing

- **Ethertype**: 0x88cc
- **Destination MAC**: 01:80:c2:00:00:0e (reserved multicast)

Q7 — LLDP TLV structure

Each LLDP frame carries TLVs (Type–Length–Value):

- Chassis ID
- Port ID
- TTL
- System Name

Each TLV: **Type (7 bits)** + **Length (9 bits)** + **Value (variable)**.

Q8 — OpenFlow packets observed

Filter `lldp` and observe:

```
PACKET_IN  : switch -> controller (new event)
PACKET_OUT : controller -> switch (action to execute)
```

Q9 — LLDP frequency

- Default: about **5 seconds**
- Higher rate: faster discovery, more overhead
- Lower rate: lower overhead, slower discovery

Q10 — How the controller discovers topology

1. ODL emits LLDP on all ports.
2. Switches send received LLDP to the controller via `PACKET_IN`.
3. The controller parses chassis/port to link nodes.
4. DLUX renders the reconstructed topology.

10 Conclusion

OpenDaylight automatically discovers the linear topology of 5 hosts and 5 switches via LLDP, while OpenFlow messages steer end-host connectivity. Wireshark validates both control and data-plane exchanges.

11 Exercise 6 — Switching Rules

Objective

Analyze flow rules installed by the SDN controller and observe OpenFlow message exchanges.

Topology

Linear with two switches:

$$h1 - s1 - s2 - h2$$

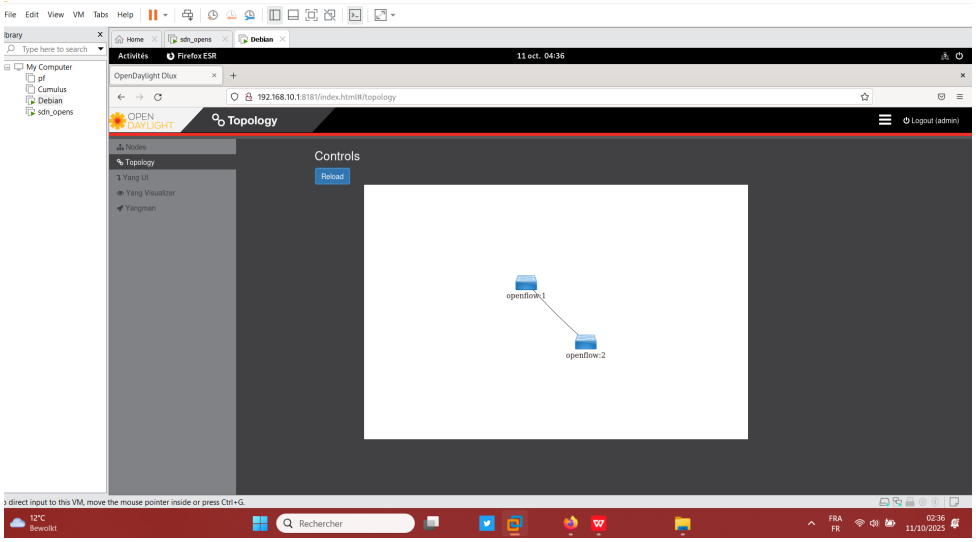

Figure 20: Topology

Create the topology

In Mininet:

```
sudo mn --topo linear,2 --controller=remote,ip=192.168.10.1,port=6653 \
  --switch ovsk,protocols=OpenFlow14
```

This forces OpenFlow 1.4 (displayed as `openflow_v4` in Wireshark).

Wireshark capture

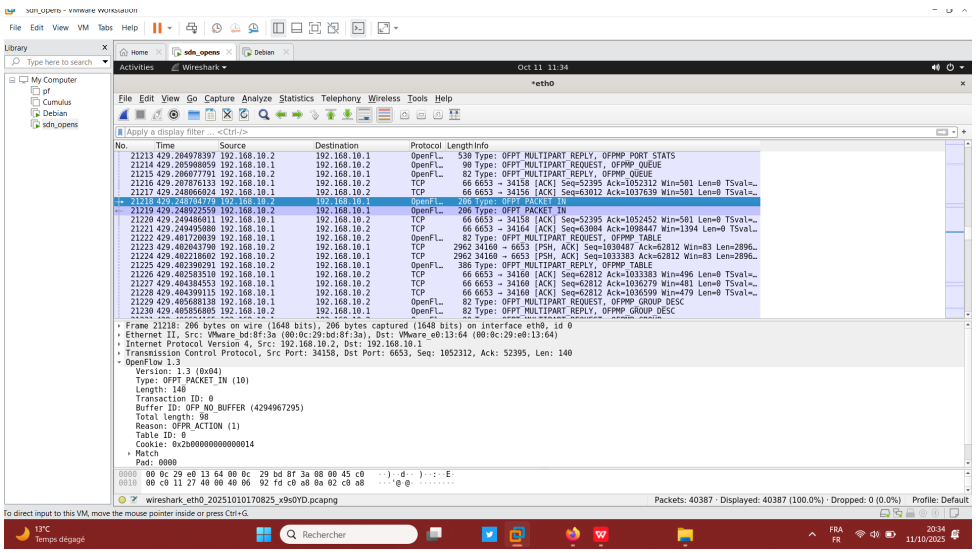

Figure 21: Wireshark capture

```
openflow_v4
```

Initial messages:

- **HELLO** — version negotiation
- **FEATURES REQUEST / REPLY** — switch capabilities
- **SET CONFIG** — controller configuration
- **PACKET_IN / PACKET_OUT** — initial unknown packets
- **FLOW_MOD** — flow installation

Type 14 filter

```
openflow_v4.type == 14
```

These are **FLOW_MOD** messages used to add/modify/delete rules.

Show rules on s2

```
sudo ovs-ofctl -O OpenFlow14 dump-flows s2
```

```
priority=0, actions=CONTROLLER:65535
```

Default rule: punt unknown packets to the controller.

OpenFlow timeouts

- **IDLE_TIMEOUT**: remove inactive rules after a period.
- **HARD_TIMEOUT**: maximum lifetime regardless of activity.

Default values: `0` (no expiry).

Ping and ICMP observation

```
icmp || openflow_v4
```

Then:

```
mininet> pingall
```

ICMP traffic appears alongside `FLOW_MOD` messages as paths are established.

New rules after ping

```
sudo ovs-ofctl -O OpenFlow14 dump-flows s2
```

Additional rules appear to forward host-to-host traffic.

Ping to a non-existent host

```
mininet> xterm h1
$ ping 10.0.0.10 -c8
```

Counters increase on the default rule:

`priority=0, actions=CONTROLLER`

Unknown traffic is punted to the controller.

Summary

- Controller and switches exchange OpenFlow messages to establish flows.
- **FLOW_MOD** (type 14) installs rules.
- Default rules punt unknown packets to the controller.
- **PACKET_IN/PACKET_OUT** bootstrap communication.
- `n_packets` counters reflect per-rule matches.

Conclusion

ODL learns the topology and dynamically installs switching rules. Each host ping triggers OpenFlow exchanges (`FLOW_MOD`, `PACKET_IN`, `PACKET_OUT`) that populate switch flow tables.